# Jason exercises: single agent

**Run command: java -jar jedit.jar &**
**from jason-2.4/jedit**

**Playing with “pseudo”-strong negation and true negation as failure**

```
/* Initial beliefs and rules */

stored(orange, 100).
stored(apple, _).
~stored(apple, _).


/* Initial goals */

!start.

/* Plans */

// +!start : stored(orange, X) <- .print("Stored orange: ", X).
// +!start : not stored(banana, X) <- .print("Bananas are not stored!").
// +!start : stored(apple, X) & ~stored(apple, X) <- .print("Apples are not stored... or maybe they are...").
+!start : stored(apple, X) & not stored(apple, X) <- .print("Apples are not stored... or maybe they are...").
```

**Playing with plans**

The final program is the following and starts from the previous exercise on fruit storage. The exercise should develop according to the following trace:

1) goals can have parameters, which can be variables. Can you write a plan for managing the
+!warehouse_display(Category)
event, and printing all the stored stuff of a given category?
**[Learning objective: using variables in goals and plans; using findall]**

2) can you write a plan that finds a subclass of the given category, whose stored amount is 0, and restores it by bringing it to the refill_amount for that category?
**[Learning objective: removing and adding beliefs]**

3) can you write a plan that restores all the stored subclasses of a given category whose amount is 0?
**[Learning objective: defining recursive plans; how to manage plan failure]**

```
/* Initial beliefs and rules */

stored(cherry, 0).
stored(banana, 10).
stored(pear, 40).
stored(apple, 50).
stored(orange, 100).
stored(lemon, 0).
stored(lime, 50).
stored(grapefruit, 400).
stored(gorgonzola, 30).

stored(parmigiano, 4).
stored(stracchino, 80).
stored(robiola, 20).
stored(mozzarella, 10).

isa(citrus, fruit).
isa(orange, citrus).
isa(lemon, citrus).
isa(lime, citrus).
isa(grapefruit, citrus).
```

```
isa(banana, fruit).
isa(apple, fruit).
isa(pear, fruit).
isa(cherry, fruit).

isa(parmigiano, cheese).
isa(stracchino, cheese).
isa(robiola, cheese).
isa(mozzarella, cheese).

refill_amount(fruit, 60).
refill_amount(cheese, 10).

stored(Superclass, Subclass, Amount) :-
transitive_isa(Subclass, Superclass) &
stored(Subclass, Amount).

transitive_isa(Subclass, Superclass) :- isa(Subclass, Superclass).
transitive_isa(Subclass, Superclass) :- isa(Subclass, BetweenClass) &
                                        transitive_isa(BetweenClass, Superclass).


/* Initial goals */

!refill(fruit).

/* Plans */

+!refill(Category) :
stored(Category, Subclass, 0) & refill_amount(Category, X)
<- .print("Warehouse situation before restorage: ");
   !warehouse_display(Category);
   .print("We must restore  ", Subclass);
   -stored(Subclass, 0); +stored(Subclass, X);
   .print("Warehouse situation after restorage: ");
   !warehouse_display(Category);
   .print("\n");
   !refill(Category).

-!refill(Category) <- .print("Restorage of  ", Category, " completed!").

+!warehouse_display(Category) : .findall(c(Subclass, Amount),
                    stored(Category, Subclass,Amount),
                                            P)
<- .print("Stored ", Category, ": ", P).
```